

sistema_experto_climatico

March 16, 2026

1 Sistema Experto - Control Climatico de Edificios

UT3T1 - Diseño e implementacion de un sistema experto con Experta

He elegido hacer un sistema que controle la climatizacion de un edificio. La idea es que reciba datos de sensores (temperatura, humedad, CO2 y ocupacion) y decida que hacer: encender calefaccion, refrigeracion, ventilar, etc. Tambien he añadido un modelo de series temporales que predice la temperatura para que el sistema pueda actuar antes de que haga falta.

1.1 Imports

```
[1]: # Por si faltaran librerias
# !pip install experta scikit-learn matplotlib pandas numpy

# Parche para que experta funcione en Python 3.10+ (sacado de https://github.
# ↪com/nilp0inter/experta/issues/55)
import collections
if not hasattr(collections, 'Mapping'):
    import collections.abc
    collections.Mapping = collections.abc.Mapping

from experta import *
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline
from datetime import datetime, timedelta
import warnings
warnings.filterwarnings('ignore')

print("Imports OK")
```

Imports OK

1.2 Hechos

Defino los hechos con Field igual que en el ejemplo de la sesion 1bis. Hay 4 tipos de sensores como entrada y luego hechos que se van creando por las reglas.

[2]: *# Hechos del sistema, como Persona y Pago de la sesion 1bis pero con sensores*

```
class SensorTemperatura(Fact):
    zona = Field(str, mandatory=True)
    valor = Field(float, mandatory=True)          # grados celsius
    timestamp = Field(str, default="2026-03-14T12:00:00")

class SensorHumedad(Fact):
    zona = Field(str, mandatory=True)
    valor = Field(float, mandatory=True)          # porcentaje 0-100

class SensorOcupacion(Fact):
    zona = Field(str, mandatory=True)
    personas = Field(int, mandatory=True)

class SensorCO2(Fact):
    zona = Field(str, mandatory=True)
    ppm = Field(float, mandatory=True)           # partes por millon

class PrediccionTemperatura(Fact):               # viene del modelo de series
    ↪ temporales
    zona = Field(str, mandatory=True)
    valor_futuro = Field(float, mandatory=True)
    horizonte_min = Field(int, default=30)

# Estos los crean las reglas, no los meto yo
class NivelConfort(Fact):                       # frio, optimo o caluroso
    zona = Field(str, mandatory=True)
    nivel = Field(str, mandatory=True)

class NivelCalidadAire(Fact):                   # bueno, regular o malo
    zona = Field(str, mandatory=True)
    nivel = Field(str, mandatory=True)

class RiesgoEnergetico(Fact):
    zona = Field(str, mandatory=True)
    nivel = Field(str, mandatory=True)

# Salida del sistema
class AccionControl(Fact):
    zona = Field(str, mandatory=True)
    sistema = Field(str, mandatory=True)        # hvac, ventilacion,
    ↪ deshumidificador
```

```

accion = Field(str, mandatory=True)
intensidad = Field(str, default="media")

class Alerta(Fact):
    zona = Field(str, mandatory=True)
    tipo = Field(str, mandatory=True)
    mensaje = Field(str, default="")
    prioridad = Field(str, default="media")

class EstadoSistema(Fact):
    modo = Field(str, default="normal")

print("Hechos definidos")

```

Hechos definidos

1.3 Motor de inferencia

Las reglas las he organizado con salience en 4 niveles, como vi en la guía técnica que el motor ejecuta primero las de mayor salience: - salience 100: emergencias (lo primero es comprobar si hay peligro) - salience 50: clasificar los datos de los sensores - salience 20: decidir acciones de control - salience 10: ajustes y optimización

En total son 17 reglas.

```

[3]: # Motor con las 17 reglas separadas por salience (100=emergencias,
    ↪ 50=clasificar, 20=control, 10=ajustes)
    # Sin el salience no funcionaba porque las de control se ejecutaban antes que
    ↪ las de clasificacion

class ControlClimatico(KnowledgeEngine):

    @DefFacts()
    def hechos_iniciales(self):
        yield EstadoSistema(modo="normal") # al hacer reset() arranca en normal

    # ---- EMERGENCIAS (salience=100) ----

    @Rule(
        SensorTemperatura(zona=MATCH.zona, valor=P(lambda t: t > 45 or t < 0)),
    ↪ # P() comprueba condicion
        salience=100
    )
    def emergencia_temp(self, zona):
        print(f" EMERGENCIA: Zona '{zona}' - temperatura extrema")
        self.declare(Alerta(zona=zona, tipo="evacuacion",
            mensaje=f"Temperatura extrema en {zona}, evacuar",
    ↪ prioridad="critica"))

```

```

        self.declare(AccionControl(zona=zona, sistema="hvac",
                                    accion="parada_emergencia", intensidad="maxima"))

    @Rule(
        SensorCO2(zona=MATCH.zona, ppm=P(lambda c: c > 5000)),
        salience=100
    )
    def emergencia_co2(self, zona):
        print(f" EMERGENCIA: Zona '{zona}' - CO2 toxico (>5000 ppm)")
        self.declare(Alerta(zona=zona, tipo="evacuacion",
                             mensaje=f"CO2 toxico en {zona}, evacuar y ventilar",
        ↪prioridad="critica"))
        self.declare(AccionControl(zona=zona, sistema="ventilacion",
                                    accion="ventilacion_emergencia", intensidad="maxima"))

    # ---- CLASIFICACION (salience=50) ----
    # Paso los numeros a categorias. NOT(Alerta critica) para no clasificar en
    ↪emergencias

    @Rule(
        SensorTemperatura(zona=MATCH.zona, valor=P(lambda t: t < 19)),
        NOT(Alerta(zona=MATCH.zona, prioridad=L("critica"))), # L() compara
    ↪valor exacto
        salience=50
    )
    def clasificar_frio(self, zona):
        print(f" Clasificacion: Zona '{zona}' -> FRIO")
        self.declare(NivelConfort(zona=zona, nivel="frio"))

    @Rule(
        SensorTemperatura(zona=MATCH.zona, valor=P(lambda t: t > 26)),
        NOT(Alerta(zona=MATCH.zona, prioridad=L("critica"))),
        salience=50
    )
    def clasificar_calor(self, zona):
        print(f" Clasificacion: Zona '{zona}' -> CALUROSO")
        self.declare(NivelConfort(zona=zona, nivel="caluroso"))

    @Rule(
        SensorTemperatura(zona=MATCH.zona, valor=P(lambda t: 19 <= t <= 26)),
        NOT(Alerta(zona=MATCH.zona, prioridad=L("critica"))),
        salience=50
    )
    def clasificar_optimo(self, zona):
        print(f" Clasificacion: Zona '{zona}' -> OPTIMO")
        self.declare(NivelConfort(zona=zona, nivel="optimo"))

```

```

@Rule(SensorCO2(zona=MATCH.zona, ppm=P(lambda c: c > 1000 and c <= 5000)),
↳salience=50)
def clasificar_aire_malo(self, zona):
    print(f" Clasificacion: Zona '{zona}' -> aire MALO")
    self.declare(NivelCalidadAire(zona=zona, nivel="malo"))

@Rule(SensorCO2(zona=MATCH.zona, ppm=P(lambda c: 400 < c <= 1000)),
↳salience=50)
def clasificar_aire_regular(self, zona):
    print(f" Clasificacion: Zona '{zona}' -> aire REGULAR")
    self.declare(NivelCalidadAire(zona=zona, nivel="regular"))

@Rule(SensorCO2(zona=MATCH.zona, ppm=P(lambda c: c <= 400)), salience=50)
def clasificar_aire_bueno(self, zona):
    print(f" Clasificacion: Zona '{zona}' -> aire BUENO")
    self.declare(NivelCalidadAire(zona=zona, nivel="bueno"))

@Rule(
    SensorOcupacion(zona=MATCH.zona, personas=P(lambda p: p > 30)),
    OR(NivelConfort(zona=MATCH.zona, nivel=L("frio")),      # OR = una u otra
        NivelConfort(zona=MATCH.zona, nivel=L("caluroso"))),
    salience=50
)
def riesgo_energetico(self, zona): # mucha gente + temp no optima = gasto
↳alto
    print(f" Clasificacion: Zona '{zona}' -> riesgo energetico ALTO")
    self.declare(RiesgoEnergetico(zona=zona, nivel="alto"))

# ---- CONTROL (salience=20) ----
# NOT(AccionControl...) para que no repita la misma accion, al principio me
↳pasaba

@Rule(
    NivelConfort(zona=MATCH.zona, nivel=L("frio")),
    SensorOcupacion(zona=MATCH.zona, personas=P(lambda p: p > 0)),
    NOT(AccionControl(zona=MATCH.zona, accion=L("calefaccion"))),
    NOT(Alerta(zona=MATCH.zona, prioridad=L("critica"))),
    salience=20
)
def activar_calefaccion(self, zona): # frio + gente = calefaccion
    print(f" Control: Zona '{zona}' -> activar CALEFACCION")
    self.declare(AccionControl(zona=zona, sistema="hvac",
        accion="calefaccion", intensidad="alta"))

@Rule(
    NivelConfort(zona=MATCH.zona, nivel=L("caluroso")),
    SensorOcupacion(zona=MATCH.zona, personas=P(lambda p: p > 0)),

```

```

        NOT(AccionControl(zona=MATCH.zona, accion=L("refrigeracion"))),
        NOT(Alerta(zona=MATCH.zona, prioridad=L("critica"))),
        salience=20
    )
    def activar_refrigeracion(self, zona): # calor + gente = aire
        print(f" Control: Zona '{zona}' -> activar REFRIGERACION")
        self.declare(AccionControl(zona=zona, sistema="hvac",
            accion="refrigeracion", intensidad="alta"))

    @Rule(
        NivelCalidadAire(zona=MATCH.zona, nivel=L("malo")),
        NOT(AccionControl(zona=MATCH.zona, accion=L("ventilacion"))),
        NOT(AccionControl(zona=MATCH.zona, accion=L("ventilacion_emergencia"))),
        salience=20
    )
    def ventilacion_forzada(self, zona): # aire malo = ventilar
        print(f" Control: Zona '{zona}' -> VENTILACION forzada")
        self.declare(AccionControl(zona=zona, sistema="ventilacion",
            accion="ventilacion", intensidad="alta"))

    @Rule(
        NivelConfort(zona=MATCH.zona, nivel=L("optimo")),
        SensorOcupacion(zona=MATCH.zona, personas=L(0)), # L(0) = exactamente 0
        NOT(AccionControl(zona=MATCH.zona)),
        salience=20
    )
    def modo_ahorro(self, zona): # nadie + temp bien = ahorrar
        print(f" Control: Zona '{zona}' -> MODO AHORRO (vacía)")
        self.declare(AccionControl(zona=zona, sistema="hvac",
            accion="modo_ahorro", intensidad="baja"))

    @Rule(
        PrediccionTemperatura(zona=MATCH.zona, valor_futuro=P(lambda t: t > 28)),
        NivelConfort(zona=MATCH.zona, nivel=L("optimo")), # ahora esta bien
        SensorOcupacion(zona=MATCH.zona, personas=P(lambda p: p > 0)),
        NOT(AccionControl(zona=MATCH.zona, accion=L("pre_enfriamiento"))),
        salience=20
    )
    def control_predictivo(self, zona): # el modelo dice que sube, enfriar ya
        print(f" Control predictivo: Zona '{zona}' -> PRE-ENFRIAMIENTO")
        self.declare(AccionControl(zona=zona, sistema="hvac",
            accion="pre_enfriamiento", intensidad="media"))

    # ---- OPTIMIZACION (salience=10) ----

```

```

@Rule(
    AS.accion << AccionControl(zona=MATCH.zona, accion=L("refrigeracion"),
↳intensidad=L("alta")),
    RiesgoEnergetico(zona=MATCH.zona, nivel=L("alto")),
    salience=10
)
def conflicto_confort_energia(self, accion, zona): # gasta mucho, bajo
↳intensidad
    print(f" Optimizacion: Zona '{zona}' -> bajar refrigeracion a
↳MODERADA")
    self.modify(accion, intensidad="moderada") # modify() como en el
↳ejemplo de prestamos

@Rule(
    SensorHumedad(zona=MATCH.zona, valor=P(lambda h: h > 70)),
    NivelConfort(zona=MATCH.zona, nivel=L("caluroso")),
    NOT(AccionControl(zona=MATCH.zona, sistema=L("deshumidificador"))),
    salience=10
)
def ajuste_humedad(self, zona): # calor + humedad = agobio
    print(f" Optimizacion: Zona '{zona}' -> activar DESHUMIDIFICADOR")
    self.declare(AccionControl(zona=zona, sistema="deshumidificador",
        accion="deshumidificar", intensidad="alta"))

@Rule(
    NivelCalidadAire(zona=MATCH.zona, nivel=L("regular")),
    SensorOcupacion(zona=MATCH.zona, personas=P(lambda p: p > 15)),
    NOT(AccionControl(zona=MATCH.zona, accion=L("ventilacion"))),
    NOT(AccionControl(zona=MATCH.zona, accion=L("ventilacion_emergencia"))),
    salience=10
)
def ventilacion_preventiva(self, zona): # aire regular + mucha gente =
↳ventilar por si acaso
    print(f" Optimizacion: Zona '{zona}' -> ventilacion preventiva")
    self.declare(AccionControl(zona=zona, sistema="ventilacion",
        accion="ventilacion", intensidad="baja"))

print("Motor definido con 17 reglas")

```

Motor definido con 17 reglas

1.4 Series temporales

Genero datos de temperatura de 7 dias (una lectura por hora) y entreno un modelo para predecir la temperatura futura. Asi el sistema puede pre-enfriar antes de que suba.

```
[4]: # Genero datos falsos de temperatura para 7 dias (1 por hora = 168 datos)
def generar_datos_temp(dias=7, seed=42):
    np.random.seed(seed)
    fechas = []
    temps = []
    inicio = datetime(2026, 3, 7, 0, 0)

    for d in range(dias):
        for h in range(24):
            fecha = inicio + timedelta(days=d, hours=h)
            base = 22.0
            ciclo = 4.0 * np.sin(2 * np.pi * (h - 8) / 24) # sube de dia, baja
            ↪de noche
            finde = -2.0 if fecha.weekday() >= 5 else 0.0 # fines de semana
            ↪baja
            tendencia = 0.3 * d # cada dia sube
            ↪un poco
            ruido = np.random.normal(0, 1.5)
            temps.append(round(base + ciclo + finde + tendencia + ruido, 1))
            fechas.append(fecha)

    return pd.DataFrame({
        'fecha': fechas, 'hora': [f.hour for f in fechas],
        'dia_semana': [f.weekday() for f in fechas],
        'temperatura': temps, 'zona': 'oficina_norte'
    })

df_temperatura = generar_datos_temp()
print(f"Datos generados: {len(df_temperatura)} registros")
print(f"Temp media: {df_temperatura['temperatura'].mean():.1f}C, min:
    ↪{df_temperatura['temperatura'].min():.1f}C, max:
    ↪{df_temperatura['temperatura'].max():.1f}C")
df_temperatura.head(10)
```

Datos generados: 168 registros

Temp media: 22.3C, min: 14.6C, max: 30.1C

```
[4]:
```

	fecha	hora	dia_semana	temperatura	zona
0	2026-03-07 00:00:00	0	5	17.3	oficina_norte
1	2026-03-07 01:00:00	1	5	15.9	oficina_norte
2	2026-03-07 02:00:00	2	5	17.0	oficina_norte
3	2026-03-07 03:00:00	3	5	18.4	oficina_norte
4	2026-03-07 04:00:00	4	5	16.2	oficina_norte
5	2026-03-07 05:00:00	5	5	16.8	oficina_norte
6	2026-03-07 06:00:00	6	5	20.4	oficina_norte
7	2026-03-07 07:00:00	7	5	20.1	oficina_norte
8	2026-03-07 08:00:00	8	5	19.3	oficina_norte

1.4.1 Entrenar el modelo

Uso regresion polinomial de grado 3. Para la hora uso codificacion ciclica (seno y coseno) porque si no el modelo no entiende que las 23h estan cerca de las 0h.

```
[5]: # Modelo de prediccion con regresion polinomial grado 3
# La hora la paso a seno/coseno para que las 23h y las 0h queden cerca
def entrenar_modelo(df):
    df_feat = df.copy()
    df_feat['hora_sin'] = np.sin(2 * np.pi * df_feat['hora'] / 24)
    df_feat['hora_cos'] = np.cos(2 * np.pi * df_feat['hora'] / 24)
    df_feat['es_finde'] = (df_feat['dia_semana'] >= 5).astype(int)

    X = df_feat[['hora_sin', 'hora_cos', 'dia_semana', 'es_finde']].values
    y = df_feat['temperatura'].values

    modelo = make_pipeline(PolynomialFeatures(degree=3, include_bias=False),
↪LinearRegression())
    modelo.fit(X, y)

    y_pred = modelo.predict(X)
    rmse = np.sqrt(np.mean((y - y_pred) ** 2))
    print(f"Modelo entrenado - RMSE: {rmse:.2f}C, R2: {modelo.score(X, y):.4f}")
    return modelo

def predecir_temp(modelo, hora, dia_semana):
    hora_sin = np.sin(2 * np.pi * hora / 24)
    hora_cos = np.cos(2 * np.pi * hora / 24)
    es_finde = 1 if dia_semana >= 5 else 0
    X = np.array([[hora_sin, hora_cos, dia_semana, es_finde]])
    return round(float(modelo.predict(X)[0]), 1)

modelo_temp = entrenar_modelo(df_temperatura)

print("\nPredicciones de ejemplo (lunes):")
for h in [8, 12, 14, 18, 22]:
    print(f" {h:02d}:00 -> {predecir_temp(modelo_temp, h, 0)}C")
```

Modelo entrenado - RMSE: 1.33C, R2: 0.8523

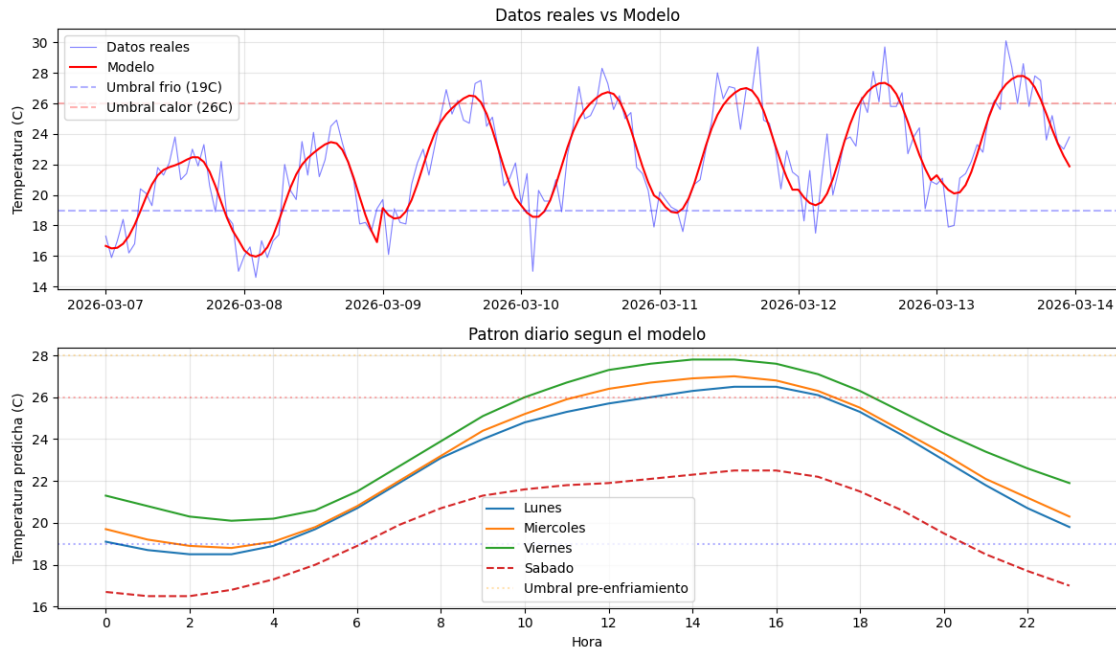
Predicciones de ejemplo (lunes):

```
08:00 -> 23.1C
12:00 -> 25.7C
14:00 -> 26.3C
18:00 -> 25.3C
22:00 -> 20.7C
```

```
[6]: # Grafico datos reales vs modelo
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 7))

ax1.plot(df_temperatura['fecha'], df_temperatura['temperatura'], 'b-', alpha=0.5, linewidth=0.8, label='Datos reales')
df_pred = df_temperatura.copy()
df_pred['hora_sin'] = np.sin(2 * np.pi * df_pred['hora'] / 24)
df_pred['hora_cos'] = np.cos(2 * np.pi * df_pred['hora'] / 24)
df_pred['es_finde'] = (df_pred['dia_semana'] >= 5).astype(int)
X_all = df_pred[['hora_sin', 'hora_cos', 'dia_semana', 'es_finde']].values
ax1.plot(df_temperatura['fecha'], modelo_temp.predict(X_all), 'r-', linewidth=1.5, label='Modelo')
ax1.axhline(y=19, color='blue', linestyle='--', alpha=0.3, label='Umbral frio (19C)')
ax1.axhline(y=26, color='red', linestyle='--', alpha=0.3, label='Umbral calor (26C)')
ax1.set_ylabel('Temperatura (C)')
ax1.set_title('Datos reales vs Modelo')
ax1.legend()
ax1.grid(True, alpha=0.3)

# Patron por dia, se ve que el sabado baja
for nombre, dia in [('Lunes', 0), ('Miercoles', 2), ('Viernes', 4), ('Sabado', 5)]:
    preds = [predecir_temp(modelo_temp, h, dia) for h in range(24)]
    ax2.plot(range(24), preds, '--' if dia >= 5 else '-', linewidth=1.5, label=nombre)
ax2.axhline(y=19, color='blue', linestyle=':', alpha=0.3)
ax2.axhline(y=26, color='red', linestyle=':', alpha=0.3)
ax2.axhline(y=28, color='orange', linestyle=':', alpha=0.3, label='Umbral pre-enfriamiento')
ax2.set_xlabel('Hora')
ax2.set_ylabel('Temperatura predicha (C)')
ax2.set_title('Patron diario segun el modelo')
ax2.set_xticks(range(0, 24, 2))
ax2.legend()
ax2.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



1.5 Ejecucion de escenarios

Funcion auxiliar para no repetir codigo cada vez que pruebo un escenario.

```
[7]: # Funcion auxiliar para probar escenarios sin repetir codigo
def ejecutar_escenario(nombre, hechos):
    print("=" * 55)
    print(f"ESCENARIO: {nombre}")
    print("=" * 55)
    engine = ControlClimatico()
    engine.reset()
    print("\nHechos de entrada:")
    for hecho in hechos:
        engine.declare(hecho)
        tipo = type(hecho).__name__
        campos = {k: v for k, v in hecho.items() if k != '__factid__'}
        print(f"  + {tipo}({','.join(f'{k}={v}' for k, v in campos.items())})")
    print("\nReglas disparadas:")
    engine.run()
    acciones = [h for h in engine.facts.values() if isinstance(h, AccionControl)]
    alertas = [h for h in engine.facts.values() if isinstance(h, Alerta)]
    print(f"\nResultado: {len(acciones)} acciones, {len(alertas)} alertas")
    for a in acciones:
        print(f"  ACCION: {a['sistema']}/{a['accion']} -> {a['intensidad']}")
    print(f"  {a['zona']}")
```

```

for a in alertas:
    print(f"  ALERTA [{a['prioridad']}]: {a['mensaje']}")
print()
return engine

```

1.6 Prueba de escenarios

1.6.1 Escenario 1: Oficina fria

Temperatura baja con gente dentro. Tiene que activar calefaccion.

```

[8]: e1 = ejecutar_escenario("Oficina en invierno", [
    SensorTemperatura(zona="oficina_norte", valor=16.5),
    SensorHumedad(zona="oficina_norte", valor=45.0),
    SensorCO2(zona="oficina_norte", ppm=650.0),
    SensorOcupacion(zona="oficina_norte", personas=12),
])

```

```

=====
ESCENARIO: Oficina en invierno
=====

```

Hechos de entrada:

```

+ SensorTemperatura(zona=oficina_norte, valor=16.5)
+ SensorHumedad(zona=oficina_norte, valor=45.0)
+ SensorCO2(zona=oficina_norte, ppm=650.0)
+ SensorOcupacion(zona=oficina_norte, personas=12)

```

Reglas disparadas:

```

Clasificacion: Zona 'oficina_norte' -> aire REGULAR
Clasificacion: Zona 'oficina_norte' -> FRIO
Control: Zona 'oficina_norte' -> activar CALEFACCION

```

Resultado: 1 acciones, 0 alertas

```

ACCION: hvac/calefaccion -> alta (oficina_norte)

```

1.6.2 Escenario 2: Emergencia

Mas de 45 grados, las reglas de emergencia se disparan primero por el salience=100.

```

[9]: e2 = ejecutar_escenario("Emergencia - temperatura extrema", [
    SensorTemperatura(zona="sala_servidores", valor=52.0),
    SensorCO2(zona="sala_servidores", ppm=800.0),
    SensorOcupacion(zona="sala_servidores", personas=2),
])

```

```

=====
ESCENARIO: Emergencia - temperatura extrema

```

=====

Hechos de entrada:

- + SensorTemperatura(zona=sala_servidores, valor=52.0)
- + SensorCO2(zona=sala_servidores, ppm=800.0)
- + SensorOcupacion(zona=sala_servidores, personas=2)

Reglas disparadas:

EMERGENCIA: Zona 'sala_servidores' - temperatura extrema
Clasificacion: Zona 'sala_servidores' -> aire REGULAR

Resultado: 1 acciones, 1 alertas

ACCION: hvac/parada_emergencia -> maxima (sala_servidores)
ALERTA [critica]: Temperatura extrema en sala_servidores, evacuar

1.6.3 Escenario 3: Control predictivo

Ahora la temp esta bien (24C) pero el modelo predice que va a subir. Deberia activar pre-enfriamiento.

```
[10]: hora_actual = 12
temp_predicha = predecir_temp(modelo_temp, hora_actual + 0.5, 0)
print(f"El modelo predice {temp_predicha}C para las 12:30")
valor_pred = max(temp_predicha, 29.5) # fuerza que sea alta para que se vea el
    ↪pre-enfriamiento

e3 = ejecutar_escenario("Control predictivo", [
    SensorTemperatura(zona="oficina_sur", valor=24.0), # ahora esta bien
    SensorCO2(zona="oficina_sur", ppm=500.0),
    SensorOcupacion(zona="oficina_sur", personas=8),
    PrediccionTemperatura(zona="oficina_sur", valor_futuro=valor_pred,
    ↪horizonte_min=30),
])
print("Sin la prediccion no se habria activado nada porque 24C esta en rango
    ↪optimo")
```

El modelo predice 25.9C para las 12:30

=====

ESCENARIO: Control predictivo

=====

Hechos de entrada:

- + SensorTemperatura(zona=oficina_sur, valor=24.0)
- + SensorCO2(zona=oficina_sur, ppm=500.0)
- + SensorOcupacion(zona=oficina_sur, personas=8)
- + PrediccionTemperatura(zona=oficina_sur, valor_futuro=29.5, horizonte_min=30)

Reglas disparadas:

Clasificacion: Zona 'oficina_sur' -> aire REGULAR
Clasificacion: Zona 'oficina_sur' -> OPTIMO
Control predictivo: Zona 'oficina_sur' -> PRE-ENFRIAMIENTO

Resultado: 1 acciones, 0 alertas

ACCION: hvac/pre_enfriamiento -> media (oficina_sur)

Sin la prediccion no se habria activado nada porque 24C esta en rango optimo

1.6.4 Escenario 4: Conflicto confort vs ahorro

Mucha gente + calor genera riesgo energetico. La regla de optimizacion baja la intensidad de la refrigeracion.

```
[11]: e4 = ejecutar_escenario("Conflicto confort vs ahorro", [  
    SensorTemperatura(zona="auditorio", valor=30.0),  
    SensorHumedad(zona="auditorio", valor=75.0),  
    SensorCO2(zona="auditorio", ppm=1200.0),  
    SensorOcupacion(zona="auditorio", personas=45),  
])
```

```
=====
```

ESCENARIO: Conflicto confort vs ahorro

```
=====
```

Hechos de entrada:

+ SensorTemperatura(zona=auditorio, valor=30.0)
+ SensorHumedad(zona=auditorio, valor=75.0)
+ SensorCO2(zona=auditorio, ppm=1200.0)
+ SensorOcupacion(zona=auditorio, personas=45)

Reglas disparadas:

Clasificacion: Zona 'auditorio' -> aire MALO
Clasificacion: Zona 'auditorio' -> CALUROSO
Clasificacion: Zona 'auditorio' -> riesgo energetico ALTO
Control: Zona 'auditorio' -> activar REFRIGERACION
Control: Zona 'auditorio' -> VENTILACION forzada
Optimizacion: Zona 'auditorio' -> bajar refrigeracion a MODERADA
Optimizacion: Zona 'auditorio' -> activar DESHUMIDIFICADOR

Resultado: 3 acciones, 0 alertas

ACCION: ventilacion/ventilacion -> alta (auditorio)
ACCION: hvac/refrigeracion -> moderada (auditorio)
ACCION: deshumidificador/deshumidificar -> alta (auditorio)

1.6.5 Escenario 5: Dos zonas distintas

Una zona fria y otra caliente. El sistema tiene que manejarlas por separado.

```
[12]: e5 = ejecutar_escenario("Dos zonas independientes", [
    SensorTemperatura(zona="planta_1", valor=15.0),
    SensorCO2(zona="planta_1", ppm=350.0),
    SensorOcupacion(zona="planta_1", personas=10),
    SensorTemperatura(zona="planta_3", valor=29.0),
    SensorCO2(zona="planta_3", ppm=900.0),
    SensorOcupacion(zona="planta_3", personas=8),
])
```

```
=====
ESCENARIO: Dos zonas independientes
=====
```

Hechos de entrada:

```
+ SensorTemperatura(zona=planta_1, valor=15.0)
+ SensorCO2(zona=planta_1, ppm=350.0)
+ SensorOcupacion(zona=planta_1, personas=10)
+ SensorTemperatura(zona=planta_3, valor=29.0)
+ SensorCO2(zona=planta_3, ppm=900.0)
+ SensorOcupacion(zona=planta_3, personas=8)
```

Reglas disparadas:

```
Clasificacion: Zona 'planta_3' -> aire REGULAR
Clasificacion: Zona 'planta_3' -> CALUROSO
Clasificacion: Zona 'planta_1' -> aire BUENO
Clasificacion: Zona 'planta_1' -> FRIO
Control: Zona 'planta_1' -> activar CALEFACCION
Control: Zona 'planta_3' -> activar REFRIGERACION
```

Resultado: 2 acciones, 0 alertas

```
ACCION: hvac/calefaccion -> alta (planta_1)
ACCION: hvac/refrigeracion -> alta (planta_3)
```

1.7 Analisis de sensibilidad

Vario la temperatura y el CO2 para ver como cambian las decisiones del sistema.

```
[13]: # Barrido de temperatura de -5 a 54 para ver que hace el sistema con cada valor
temperaturas = list(range(-5, 55, 1))
resultados_sens = []
for temp in temperaturas:
    engine = ControlClimatico()
    engine.reset()
    engine.declare(SensorTemperatura(zona="test", valor=float(temp)))
```

```

engine.declare(SensorCO2(zona="test", ppm=500.0))
engine.declare(SensorOcupacion(zona="test", personas=10))
engine.run()
acciones = [h for h in engine.facts.values() if isinstance(h,
↳ AccionControl)]
alertas = [h for h in engine.facts.values() if isinstance(h, Alerta)]
confort = [h for h in engine.facts.values() if isinstance(h, NivelConfort)]
resultados_sens.append({
    'temperatura': temp,
    'confort': confort[0]['nivel'] if confort else 'N/A',
    'accion_principal': acciones[0]['accion'] if acciones else 'ninguna',
    'emergencia': any(a['prioridad'] == 'critica' for a in alertas)
})
df_sens = pd.DataFrame(resultados_sens)

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 6))
colores = {'frio': 'blue', 'optimo': 'green', 'caluroso': 'red', 'N/A': 'gray'}
for nivel, color in colores.items():
    mask = df_sens['confort'] == nivel
    ax1.scatter(df_sens.loc[mask, 'temperatura'], [1]*mask.sum(), c=color,
↳ label=nivel, s=40)
ax1.axvline(x=0, color='purple', linestyle='--', alpha=0.4)
ax1.axvline(x=19, color='blue', linestyle='--', alpha=0.4)
ax1.axvline(x=26, color='red', linestyle='--', alpha=0.4)
ax1.axvline(x=45, color='purple', linestyle='--', alpha=0.4)
ax1.set_xlabel('Temperatura (C)')
ax1.set_title('Nivel de confort segun temperatura')
ax1.legend()
ax1.set_yticks([])
ax1.grid(True, alpha=0.3)

colores_acc = {'calefaccion': 'blue', 'ninguna': 'lightgray', 'refrigeracion':
↳ 'red', 'parada_emergencia': 'purple'}
for acc in df_sens['accion_principal'].unique():
    mask = df_sens['accion_principal'] == acc
    ax2.scatter(df_sens.loc[mask, 'temperatura'], [1]*mask.sum(), c=colores_acc.
↳ get(acc, 'gray'), label=acc, s=40)
ax2.set_xlabel('Temperatura (C)')
ax2.set_title('Accion de control segun temperatura')
ax2.legend()
ax2.set_yticks([])
ax2.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print("\nTransiciones del sistema:")
print("-" * 55)

```

```

for (low, high), label in [((-5,-1), 'T < 0C'), ((0,18), '0-18C'),
↳((19,26), '19-26C'), ((27,44), '27-44C'), ((45,54), 'T > 45C')]:
    sub = df_sens[(df_sens['temperatura'] >= low) & (df_sens['temperatura'] <=
↳high)]
    if len(sub) > 0:
        c = sub['confort'].mode().iloc[0]
        a = sub['accion_principal'].mode().iloc[0]
        e = 'SI' if sub['emergencia'].any() else 'NO'
        print(f" {label:>8} -> confort={c}, accion={a}, emergencia={e}")

```

```

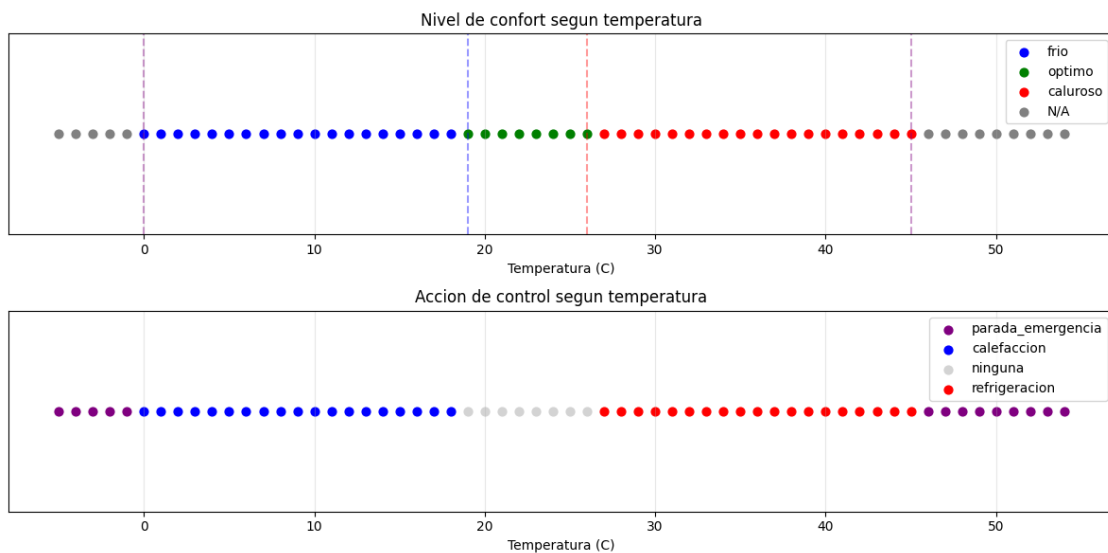
EMERGENCIA: Zona 'test' - temperatura extrema
Clasificacion: Zona 'test' -> aire REGULAR
EMERGENCIA: Zona 'test' - temperatura extrema
Clasificacion: Zona 'test' -> aire REGULAR
EMERGENCIA: Zona 'test' - temperatura extrema
Clasificacion: Zona 'test' -> aire REGULAR
EMERGENCIA: Zona 'test' - temperatura extrema
Clasificacion: Zona 'test' -> aire REGULAR
EMERGENCIA: Zona 'test' - temperatura extrema
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> FRIO
Control: Zona 'test' -> activar CALEFACCION
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> FRIO
Control: Zona 'test' -> activar CALEFACCION
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> FRIO
Control: Zona 'test' -> activar CALEFACCION
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> FRIO
Control: Zona 'test' -> activar CALEFACCION
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> FRIO
Control: Zona 'test' -> activar CALEFACCION
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> FRIO
Control: Zona 'test' -> activar CALEFACCION
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> FRIO
Control: Zona 'test' -> activar CALEFACCION
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> FRIO
Control: Zona 'test' -> activar CALEFACCION

```

Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> FRIO
Control: Zona 'test' -> activar CALEFACCION
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> FRIO
Control: Zona 'test' -> activar CALEFACCION
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> FRIO
Control: Zona 'test' -> activar CALEFACCION
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> FRIO
Control: Zona 'test' -> activar CALEFACCION
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> FRIO
Control: Zona 'test' -> activar CALEFACCION
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> FRIO
Control: Zona 'test' -> activar CALEFACCION
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> FRIO
Control: Zona 'test' -> activar CALEFACCION
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> OPTIMO
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> OPTIMO
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> OPTIMO
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> OPTIMO
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> OPTIMO
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> OPTIMO
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> CALUROSO

[illegible]

Control: Zona 'test' -> activar REFRIGERACION
 Clasificacion: Zona 'test' -> aire REGULAR
 Clasificacion: Zona 'test' -> CALUROSO
 Control: Zona 'test' -> activar REFRIGERACION
 Clasificacion: Zona 'test' -> aire REGULAR
 Clasificacion: Zona 'test' -> CALUROSO
 Control: Zona 'test' -> activar REFRIGERACION
 EMERGENCIA: Zona 'test' - temperatura extrema
 Clasificacion: Zona 'test' -> aire REGULAR
 EMERGENCIA: Zona 'test' - temperatura extrema
 Clasificacion: Zona 'test' -> aire REGULAR
 EMERGENCIA: Zona 'test' - temperatura extrema
 Clasificacion: Zona 'test' -> aire REGULAR
 EMERGENCIA: Zona 'test' - temperatura extrema
 Clasificacion: Zona 'test' -> aire REGULAR
 EMERGENCIA: Zona 'test' - temperatura extrema
 Clasificacion: Zona 'test' -> aire REGULAR
 EMERGENCIA: Zona 'test' - temperatura extrema
 Clasificacion: Zona 'test' -> aire REGULAR
 EMERGENCIA: Zona 'test' - temperatura extrema
 Clasificacion: Zona 'test' -> aire REGULAR
 EMERGENCIA: Zona 'test' - temperatura extrema
 Clasificacion: Zona 'test' -> aire REGULAR



Transiciones del sistema:

```

T < 0C -> confort=N/A, accion=parada_emergencia, emergencia=SI
0-18C -> confort=frio, accion=calefaccion, emergencia=NO
19-26C -> confort=optimo, accion=ninguna, emergencia=NO
27-44C -> confort=caluroso, accion=refrigeracion, emergencia=NO
T > 45C -> confort=N/A, accion=parada_emergencia, emergencia=SI

```

```

[14]: # Lo mismo con CO2 (temperatura fija en 22 para aislar el efecto)
niveles_co2 = list(range(200, 6000, 100))
res_co2 = []
for co2 in niveles_co2:
    engine = ControlClimatico()
    engine.reset()
    engine.declare(SensorTemperatura(zona="test", valor=22.0))
    engine.declare(SensorCO2(zona="test", ppm=float(co2)))
    engine.declare(SensorOcupacion(zona="test", personas=10))
    engine.run()
    acciones = [h for h in engine.facts.values() if isinstance(h, AccionControl)]
    alertas = [h for h in engine.facts.values() if isinstance(h, Alerta)]
    calidad = [h for h in engine.facts.values() if isinstance(h, NivelCalidadAire)]
    res_co2.append({
        'co2': co2,
        'calidad_aire': calidad[0]['nivel'] if calidad else 'N/A',
        'ventilacion': any(a['accion'] in ('ventilacion', 'ventilacion_emergencia') for a in acciones),
        'emergencia': any(a['prioridad'] == 'critica' for a in alertas)
    })
df_co2 = pd.DataFrame(res_co2)

print("Respuesta del sistema segun nivel de CO2:")
print("-" * 50)
for (low, high), label in [((200,400), '<=400 ppm'), ((401,1000), '401-1000 ppm'), ((1001,5000), '1001-5000 ppm'), ((5001,6000), '>5000 ppm')]:
    sub = df_co2[(df_co2['co2'] >= low) & (df_co2['co2'] <= high)]
    if len(sub) > 0:
        cal = sub['calidad_aire'].mode().iloc[0]
        vent = 'SI' if sub['ventilacion'].any() else 'NO'
        emerg = 'SI' if sub['emergencia'].any() else 'NO'
        print(f" {label:>15} -> calidad={cal}, ventilacion={vent}, emergencia={emerg}")

```

```

Clasificacion: Zona 'test' -> aire BUENO
Clasificacion: Zona 'test' -> OPTIMO
Clasificacion: Zona 'test' -> aire BUENO
Clasificacion: Zona 'test' -> OPTIMO
Clasificacion: Zona 'test' -> aire BUENO
Clasificacion: Zona 'test' -> OPTIMO

```

[illegible]

Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada
Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada
Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada
Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada
Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada
Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada
Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada
Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada
Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada
Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada
Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada
Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada

```

Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada
Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada
Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada
Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada
Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada
Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada
Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada
Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada
Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada
Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada
Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada
EMERGENCIA: Zona 'test' - CO2 toxico (>5000 ppm)
EMERGENCIA: Zona 'test' - CO2 toxico (>5000 ppm)
EMERGENCIA: Zona 'test' - CO2 toxico (>5000 ppm)
EMERGENCIA: Zona 'test' - CO2 toxico (>5000 ppm)
EMERGENCIA: Zona 'test' - CO2 toxico (>5000 ppm)
EMERGENCIA: Zona 'test' - CO2 toxico (>5000 ppm)
EMERGENCIA: Zona 'test' - CO2 toxico (>5000 ppm)
EMERGENCIA: Zona 'test' - CO2 toxico (>5000 ppm)
EMERGENCIA: Zona 'test' - CO2 toxico (>5000 ppm)
Respuesta del sistema segun nivel de CO2:
-----
<=400 ppm -> calidad=bueno, ventilacion=NO, emergencia=NO

```

401-1000 ppm -> calidad=regular, ventilacion=NO, emergencia=NO
 1001-5000 ppm -> calidad=malo, ventilacion=SI, emergencia=NO
 >5000 ppm -> calidad=N/A, ventilacion=SI, emergencia=SI

1.8 Tests

12 tests que comprueban los distintos comportamientos del sistema.

```
[15]: # 12 tests, cada uno mete datos y comprueba que la salida sea la esperada
def hacer_test(test_id, nombre, hechos, comprobar):
    engine = ControlClimatico()
    engine.reset()
    for h in hechos:
        engine.declare(h)
    engine.run()
    ok, msg = comprobar(engine)
    print(f" [{ 'PASS' if ok else 'FAIL' }] {test_id}: {nombre}")
    if not ok: print(f"          -> {msg}")
    return ok, test_id, nombre

print("Ejecutando tests...")
print("=" * 55)
resultados = []

# T01: frio -> calefaccion
ok, tid, tn = hacer_test("T01", "Temp baja activa calefaccion",
    [SensorTemperatura(zona="test", valor=15.0), SensorCO2(zona="test", ppm=500.
    ↪0), SensorOcupacion(zona="test", personas=5)],
    lambda e: (any(isinstance(h, AccionControl) and h['accion'] ==_
    ↪'calefaccion' for h in e.facts.values()), "No se activo la calefaccion"))
resultados.append((ok, tid, tn))

# T02: calor -> aire
ok, tid, tn = hacer_test("T02", "Temp alta activa refrigeracion",
    [SensorTemperatura(zona="test", valor=32.0), SensorCO2(zona="test", ppm=500.
    ↪0), SensorOcupacion(zona="test", personas=3)],
    lambda e: (any(isinstance(h, AccionControl) and h['accion'] ==_
    ↪'refrigeracion' for h in e.facts.values()), "No se activo la refrigeracion"))
resultados.append((ok, tid, tn))

# T03: CO2 alto -> ventilacion
ok, tid, tn = hacer_test("T03", "CO2 alto activa ventilacion",
    [SensorTemperatura(zona="test", valor=22.0), SensorCO2(zona="test",_
    ↪ppm=1500.0), SensorOcupacion(zona="test", personas=5)],
    lambda e: (any(isinstance(h, AccionControl) and h['accion'] ==_
    ↪'ventilacion' for h in e.facts.values()), "No se activo ventilacion"))
resultados.append((ok, tid, tn))
```

```

# T04: todo bien -> no hace nada
ok, tid, tn = hacer_test("T04", "Condiciones optimas, sin accion HVAC",
    [SensorTemperatura(zona="test", valor=22.0), SensorCO2(zona="test", ppm=400.
    ↪0), SensorOcupacion(zona="test", personas=5)],
    lambda e: (any(isinstance(h, NivelConfort) and h['nivel'] == 'optimo' for h
    ↪in e.facts.values()) and
        not any(isinstance(h, AccionControl) and h['accion'] in ('calefaccion',
    ↪'refrigeracion') for h in e.facts.values()),
        "Deberia estar en optimo sin acciones de climatizacion"))
resultados.append((ok, tid, tn))

# T05: cadena sensor -> clasifica frio -> calefaccion
ok, tid, tn = hacer_test("T05", "Inferencia multinivel: sensor -> confort ->
    ↪accion",
    [SensorTemperatura(zona="test", valor=15.0), SensorCO2(zona="test", ppm=500.
    ↪0), SensorOcupacion(zona="test", personas=3)],
    lambda e: (any(isinstance(h, NivelConfort) and h['nivel'] == 'frio' for h
    ↪in e.facts.values()) and
        any(isinstance(h, AccionControl) and h['accion'] == 'calefaccion' for h
    ↪in e.facts.values()),
        "Deberia haber NivelConfort(frio) y luego calefaccion"))
resultados.append((ok, tid, tn))

# T06: mucha gente + calor -> refrigeracion moderada (por riesgo energetico)
ok, tid, tn = hacer_test("T06", "Riesgo energetico reduce intensidad",
    [SensorTemperatura(zona="test", valor=30.0), SensorCO2(zona="test", ppm=500.
    ↪0), SensorOcupacion(zona="test", personas=50)],
    lambda e: (any(isinstance(h, RiesgoEnergetico) for h in e.facts.values())
    ↪and
        any(isinstance(h, AccionControl) and h['accion'] == 'refrigeracion' and
    ↪h['intensidad'] == 'moderada' for h in e.facts.values()),
        "Deberia haber riesgo energetico y refrigeracion moderada"))
resultados.append((ok, tid, tn))

# T07: prediccion alta -> pre-enfriamiento
ok, tid, tn = hacer_test("T07", "Prediccion activa pre-enfriamiento",
    [SensorTemperatura(zona="test", valor=24.0), SensorCO2(zona="test", ppm=500.
    ↪0),
    SensorOcupacion(zona="test", personas=8),
    ↪PrediccionTemperatura(zona="test", valor_futuro=30.0, horizonte_min=30)],
    lambda e: (any(isinstance(h, AccionControl) and h['accion'] ==
    ↪'pre_enfriamiento' for h in e.facts.values()), "No se activo el
    ↪pre-enfriamiento"))
resultados.append((ok, tid, tn))

```

```

# T08: 50 grados -> emergencia
ok, tid, tn = hacer_test("T08", "Emergencia tiene prioridad",
    [SensorTemperatura(zona="test", valor=50.0), SensorCO2(zona="test", ppm=500.
    ↪0), SensorOcupacion(zona="test", personas=5)],
    lambda e: (any(isinstance(h, Alerta) and h['prioridad'] == 'critica' for h
    ↪in e.facts.values()), "No se genero alerta critica"))
resultados.append((ok, tid, tn))

# T09: nadie + temp bien -> modo ahorro
ok, tid, tn = hacer_test("T09", "Modo ahorro en zona vacia",
    [SensorTemperatura(zona="test", valor=22.0), SensorCO2(zona="test", ppm=350.
    ↪0), SensorOcupacion(zona="test", personas=0)],
    lambda e: (any(isinstance(h, AccionControl) and h['accion'] ==
    ↪'modo_ahorro' for h in e.facts.values()), "No se activo modo ahorro"))
resultados.append((ok, tid, tn))

# T10: justo 19.0 = optimo (limite del rango)
ok, tid, tn = hacer_test("T10", "Umbral exacto: 19.0C es optimo",
    [SensorTemperatura(zona="test", valor=19.0), SensorCO2(zona="test", ppm=500.
    ↪0), SensorOcupacion(zona="test", personas=5)],
    lambda e: (any(isinstance(h, NivelConfort) and h['nivel'] == 'optimo' for h
    ↪in e.facts.values()), "19.0 deberia ser optimo"))
resultados.append((ok, tid, tn))

# T11: solo ocupacion sin temperatura -> no peta ni hace nada raro
ok, tid, tn = hacer_test("T11", "Sin sensores de temp no hace nada raro",
    [SensorOcupacion(zona="test", personas=5)],
    lambda e: (not any(isinstance(h, AccionControl) for h in e.facts.values())
    ↪and
    ↪not any(isinstance(h, Alerta) for h in e.facts.values()), "No deberia
    ↪generar acciones sin temperatura"))
resultados.append((ok, tid, tn))

# T12: zona A fria + zona B caliente = calefaccion en A y aire en B
ok, tid, tn = hacer_test("T12", "Dos zonas funcionan independientes",
    [SensorTemperatura(zona="zona_A", valor=15.0), SensorCO2(zona="zona_A",
    ↪ppm=500.0), SensorOcupacion(zona="zona_A", personas=5),
    ↪SensorTemperatura(zona="zona_B", valor=30.0), SensorCO2(zona="zona_B",
    ↪ppm=500.0), SensorOcupacion(zona="zona_B", personas=5)],
    lambda e: (any(isinstance(h, AccionControl) and h['zona'] == 'zona_A' and
    ↪h['accion'] == 'calefaccion' for h in e.facts.values()) and
    ↪any(isinstance(h, AccionControl) and h['zona'] == 'zona_B' and
    ↪h['accion'] == 'refrigeracion' for h in e.facts.values()),
    ↪"Deberia haber calefaccion en A y refrigeracion en B"))
resultados.append((ok, tid, tn))

```

```

print("\n" + "=" * 55)
pasados = sum(1 for ok, _, _ in resultados if ok)
print(f"Resultado: {pasados}/{len(resultados)} tests pasados")
for ok, tid, nombre in resultados:
    print(f"  {'PASS' if ok else 'FAIL'} - {tid}: {nombre}")

```

Ejecutando tests...

```

=====
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> FRIO
Control: Zona 'test' -> activar CALEFACCION
[PASS] T01: Temp baja activa calefaccion
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> CALUROSO
Control: Zona 'test' -> activar REFRIGERACION
[PASS] T02: Temp alta activa refrigeracion
Clasificacion: Zona 'test' -> aire MALO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> VENTILACION forzada
[PASS] T03: CO2 alto activa ventilacion
Clasificacion: Zona 'test' -> aire BUENO
Clasificacion: Zona 'test' -> OPTIMO
[PASS] T04: Condiciones optimas, sin accion HVAC
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> FRIO
Control: Zona 'test' -> activar CALEFACCION
[PASS] T05: Inferencia multinivel: sensor -> confort -> accion
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> CALUROSO
Clasificacion: Zona 'test' -> riesgo energetico ALTO
Control: Zona 'test' -> activar REFRIGERACION
Optimizacion: Zona 'test' -> bajar refrigeracion a MODERADA
Optimizacion: Zona 'test' -> ventilacion preventiva
[PASS] T06: Riesgo energetico reduce intensidad
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> OPTIMO
Control predictivo: Zona 'test' -> PRE-ENFRIAMIENTO
[PASS] T07: Prediccion activa pre-enfriamiento
EMERGENCIA: Zona 'test' - temperatura extrema
Clasificacion: Zona 'test' -> aire REGULAR
[PASS] T08: Emergencia tiene prioridad
Clasificacion: Zona 'test' -> aire BUENO
Clasificacion: Zona 'test' -> OPTIMO
Control: Zona 'test' -> MODO AHORRO (vacía)
[PASS] T09: Modo ahorro en zona vacía
Clasificacion: Zona 'test' -> aire REGULAR
Clasificacion: Zona 'test' -> OPTIMO
[PASS] T10: Umbral exacto: 19.0C es optimo

```

```
[PASS] T11: Sin sensores de temp no hace nada raro
Clasificacion: Zona 'zona_B' -> aire REGULAR
Clasificacion: Zona 'zona_B' -> CALUROSO
Clasificacion: Zona 'zona_A' -> aire REGULAR
Clasificacion: Zona 'zona_A' -> FRIO
Control: Zona 'zona_A' -> activar CALEFACCION
Control: Zona 'zona_B' -> activar REFRIGERACION
[PASS] T12: Dos zonas funcionan independientes
```

```
=====
Resultado: 12/12 tests pasados
```

```
PASS - T01: Temp baja activa calefaccion
PASS - T02: Temp alta activa refrigeracion
PASS - T03: CO2 alto activa ventilacion
PASS - T04: Condiciones optimas, sin accion HVAC
PASS - T05: Inferencia multinivel: sensor -> confort -> accion
PASS - T06: Riesgo energetico reduce intensidad
PASS - T07: Prediccion activa pre-enfriamiento
PASS - T08: Emergencia tiene prioridad
PASS - T09: Modo ahorro en zona vacia
PASS - T10: Umbral exacto: 19.0C es optimo
PASS - T11: Sin sensores de temp no hace nada raro
PASS - T12: Dos zonas funcionan independientes
```

1.9 Conclusiones

El sistema funciona bien en todos los escenarios que he probado. Lo que mas me ha costado entender es que el orden de las reglas importa mucho, sin el salience las reglas de control se ejecutaban antes que las de clasificacion y no funcionaba.

Tambien tuve problemas con las acciones duplicadas hasta que use NOT() para comprobar que no existiera ya la accion antes de crearla.

La parte de series temporales esta bien porque permite que el sistema actue de forma preventiva en vez de solo reactiva. Solo predice temperatura pero se podria hacer lo mismo con el CO2 o la ocupacion.